

RapidBoxForest: Link-Envelope-Guided Box Forests for Fast Multi-Query Manipulation

TIAN, Yu and Ren, Hongliang

Abstract—Fast repeated-query manipulation benefits from reusable volumetric planning structure, but high-quality IRIS-style convex regions are expensive to construct and rebuild. This paper presents RapidBoxForest (RBF), a link-envelope-guided box-forest planner for fast multi-query manipulation. The central geometric primitive maps endpoint interval envelopes to link interval envelopes via capsule Minkowski decomposition and convex-hull generator bounding applied to *C-space box validation and multi-query reuse*: endpoint interval envelopes bound rounded-link endpoint motion over a joint box, and a radius-lifted link envelope provides an inexpensive filter for box validation and collision rejection. A rapidly-exploring random tree (RRT)-inspired grower builds a scene-specific box forest into a queryable corridor graph, and a Lifelong Envelope Cache Tree (LECT) selectively caches kinematics-keyed envelope records when replay is cheaper than recomputation. The reported experiments evaluate strict-audit Shelf+IIWA planning, the link-envelope representation trade-off, persistent evidence replay, and local dynamic-scene updates. On Shelf+IIWA, warm external-evidence rows preserve strict-audit success, while wall time remains dominated by scene-specific growth and validation in the inexpensive LinkIAABB setting. The dynamic-update study reports the accounting needed to exploit the leaf-sweep obstacle relation cache under matched obstacle-edit paths. This paper keeps the original planning formulation and title, but limits its quantitative claims to results regenerated from the current RapidBoxForest implementation.

Index Terms—Motion planning, multi-query planning, link interval envelopes, box-region planning, final audit.

I. Introduction

Motion planning for high-degree-of-freedom (DOF) manipulators becomes especially demanding when a system must answer many related queries rather than one isolated instance. In warehouses, packing cells, and shared workcells, robot kinematics stay fixed while obstacles, task endpoints, or local scene edits change. The central question is how to construct reusable *C-space* planning structure quickly enough that repeated queries can benefit from region-level search without paying a large scene-specific preprocessing cost.

Iterative Regional Inflation by Semidefinite Programming (IRIS) and Graph-of-Convex-Sets (GCS) methods

show why free-region construction is valuable: once good convex regions are available, graph search or convex optimization can produce high-quality motions over those regions [1], [2], [3]. The limitation is construction cost. In high-dimensional manipulator scenes, inflating and validating high-quality convex regions can dominate the planning budget and must often be repeated when the scene changes. Sampling-based planners such as probabilistic roadmap (PRM) and rapidly-exploring random tree (RRT) methods avoid this region-construction cost [4], [5], [6], [7], but their reusable structures are usually zero-volume graphs, so repeated queries still pay for local collision checking and do not obtain a reusable free-region cover.

RBF targets a different design point by constructing simpler box regions quickly. Instead of inflating general polytopes, it grows axis-aligned boxes in configuration space, connects accepted boxes into a graph, and answers later queries by searching the resulting box corridor. This trades per-region expressivity and sometimes path length for cheap construction, simple adjacency, and direct reuse across many queries. The intended claim is a system-level one: a fast box-region planning pipeline can be a useful final-audited alternative when multi-query latency matters more than obtaining the shortest path from a more expensive region decomposition.

The key technical ingredient is a link interval envelope computed from endpoint interval envelopes. For rounded link models, RBF reduces link motion over a joint box to the motion of a zero-radius centerline plus a final radius lift. Endpoint interval envelopes bound the endpoint trajectories over the joint box; link interval envelopes then enclose the swept rounded link and can be stored as axis-aligned bounding boxes (AABBs), *k*-DOP slabs, or SupportHull records. Strict interval sources provide an optional conservative validation path, while tighter practical sources are used as fast filters and discharged by the same final scene audit used for all reported planners.

On top of this geometric primitive, the grower borrows the rapid frontier-expansion idea of RRT: samples choose useful frontiers, a local region oracle materializes a free box around the projected seed, and accepted boxes extend a connected box forest. The result is also an adaptive cell-decomposition view of manipulator *C-space*, but one biased toward the regions needed by the current repeated-query workload. A Lifelong Envelope Cache Tree

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

(LECT) then memoizes scene-independent endpoint and link-envelope evidence keyed by robot kinematics and joint intervals; it is an acceleration layer for repeated materialization, not a separate planning claim.

The evaluation therefore asks what the current implementation can support under the adopted protocol. The main axes are Shelf+IIWA strict-audit performance, evidence replay in the active leaf-refine ablation, envelope-representation trade-offs across the link-envelope families including staged AABB/KDOP26/SupportHull cascades, and dynamic-update accounting for the leaf-sweep-RRT planner. Broader cross-planner baselines and full multi-robot generalization remain future work.

We therefore position the paper as a planning-systems contribution enabled by a geometry primitive. The endpoint-to-link envelope is the main geometric idea; the method shows how that primitive supports a reproducible low-build-cost multi-query design point under final audit.

The paper’s main contribution is a reproducible final-audited multi-query planning system and its measured design point. It is supported by three concrete elements:

- 1) *An endpoint-to-link envelope primitive for C-space box validation.* Capsule Minkowski decomposition and convex-hull generator bounding are individually classical facts already combined in articulated-body CCD work [8], [9], [10]. RBF applies this two-step pipeline to *C-space box growth and multi-query reuse*: a switchable endpoint source family—AA-backed interval forward kinematics (IFK), hierarchical IFK (HIFK), CritSample, analytical extrema—compact per-representation link records (AABB, KDOP26, SupportHull) for staged filtering, and a kinematic-keyed LECT cache for cross-scene envelope reuse. No prior work has been found applying this pipeline to *C-space box validation* or *multi-query planning reuse* (sections III and VI-C).
- 2) *A self-contained link-envelope-guided box-forest planner.* Scene-specific *C-space box forests* are grown through seeded frontier expansion, consolidated into a connected corridor graph, and queried by graph search and local refinement. LECT provides an optional kinematic-keyed cache for repeated envelope materialization across builds (sections IV, V and VI-C).
- 3) *An implementation-aligned evaluation.* Shelf+IIWA, envelope sweep, reduced random-scene, and dynamic-update studies are evaluated under the strict audit protocol adopted in this paper. The reported rows are generated from the current implementation and do not carry forward the older cross-planner comparison tables (sections VI-A, VI-C and VI-D).

The paper’s evidence chain is: Section III develops the endpoint and link interval-envelope primitive for fast box validation; section IV describes the cache optimization that reuses expensive envelope records; section V turns link-envelope-filtered boxes into a rapidly expanding box-forest planner; and section VI evaluates the resulting

build/query, cache-reuse, and dynamic-update behavior under the adopted protocol.

II. Related Work

A. Convex Region Planning and Graphs of Convex Sets

Convex region planners approach motion planning by constructing a certified cover of free space and then solving graph search or convex optimization on top of that cover. IRIS introduced the now-standard idea of computing large collision-free convex regions by alternating separating hyperplanes and region inflation [1]. Later work extended the same line into configuration space, improved region quality, or improved graph compactness through visibility-graph and clique-cover constructions [11], [12], [13]. Marucci *et al.* made the Graph-of-Convex-Sets formulation particularly influential by showing that motion planning can be cast as optimization over a graph of certified convex regions [2], [3].

RBF is aligned with this family in objective but not in geometric primitive: it uses axis-aligned boxes in *C-space* rather than general polytopes, trading per-region expressivity for lower build overhead and simpler adjacency bookkeeping. The resulting cover is coarser than a high-quality convex decomposition, but it avoids repeated hyperplane-based region inflation and is easier to rebuild under repeated-query workloads. The comparison in this paper therefore concerns fast reusable free-region construction versus tighter, more scene-coupled regional convexification, rather than box and polytope representations in isolation.

B. Swept Volume Approximation and Capsule Geometry

Computing the swept volume of a moving rigid body has been studied extensively in computational geometry and computer-aided manufacturing (CAM) communities [14], [15]. For a convex body $P = \text{conv}(v_1, \dots, v_m)$, bounding each generator trajectory by a conservative outer set $\hat{\Gamma}_i(I)$ yields

$$S_I(P) \subseteq \text{conv}\left(\bigcup_i \hat{\Gamma}_i(I)\right),$$

a bound that is implicit in oriented bounding box (OBB) tree construction for swept mesh primitives [16].

For capsule links this bound reduces to bounding only the two segment endpoints. The Proximity Query Package (PQP) exploits exactly this Minkowski decomposition ($C = S \oplus B_2(r)$, sweep of C reduces to sweep of S plus radius lift) [17]; GJK and its robust implementation by van den Bergen apply the same identity for distance queries [18], [19]; the Flexible Collision Library (FCL) generalises the swept-sphere interface to a full collision library [20].

C. Interval Forward Kinematics and the CCD Endpoint-Bounding Pipeline

Interval arithmetic [21], [22] provides inclusion-monotone enclosures of real-valued functions over

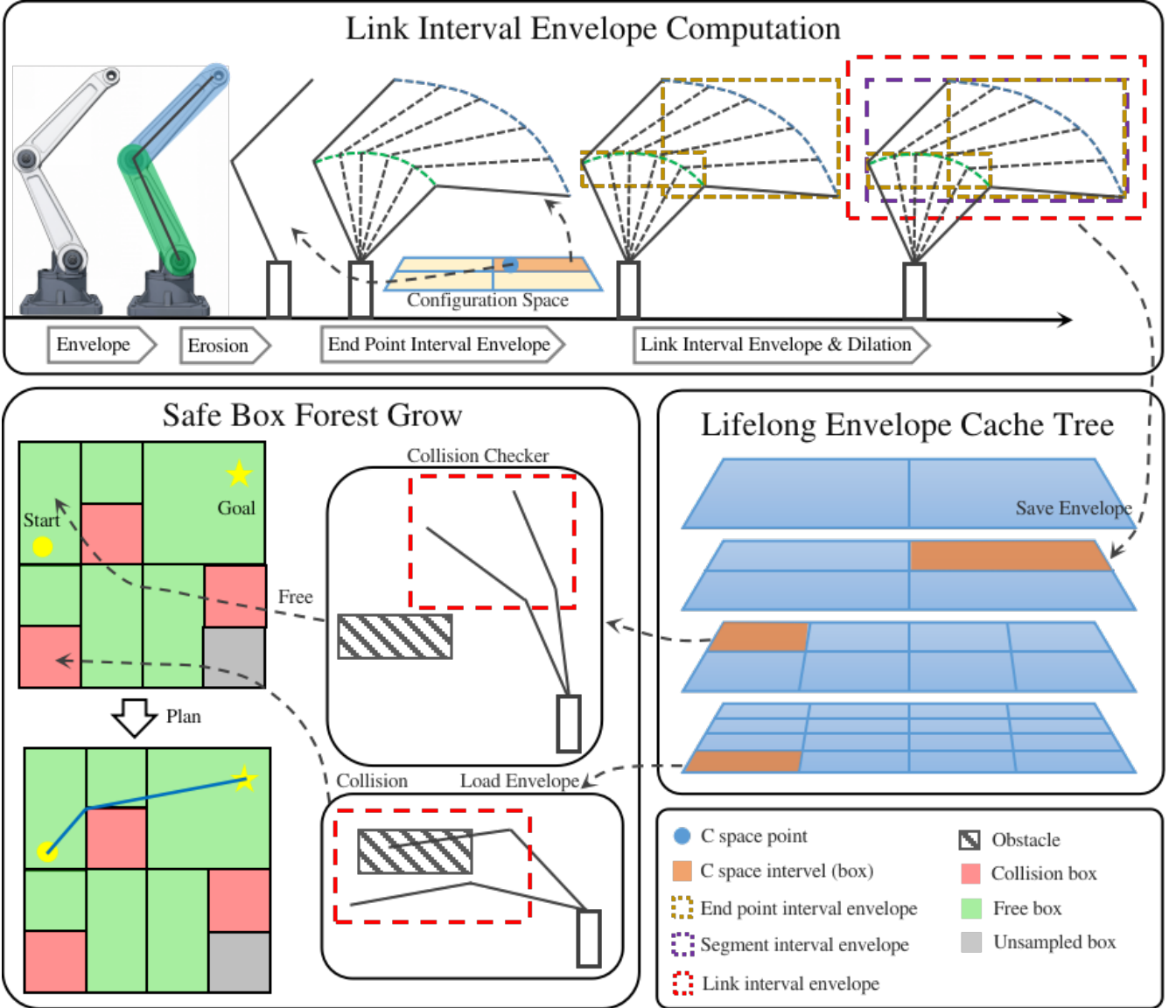


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a C -space box. Middle: LECT optionally reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and an RRT-inspired grower expand a connected box-region forest for lightweight corridor queries and final audit.

parameter intervals. Snyder applied interval arithmetic to parametric geometry to compute conservative bounding boxes for curves and surfaces without mesh sampling [23]. Merlet extended DH-chain interval propagation to certified workspace analysis for parallel and serial manipulators [24], [25].

These ideas meet directly in the continuous collision detection (CCD) literature for articulated bodies. Redon, Kim, Lin, and Manocha [8] model each robot limb as a line-swept sphere (LSS, i.e. capsule) and propagate interval Denavit-Hartenberg (DH) transforms along the kinematic chain to obtain *interval vectors bounding the two LSS endpoint positions* over the motion time step; the swept capsule is then bounded by padding the endpoint bounding box with the link radius. This is explicitly the two-step pipeline

$$\text{interval FK endpoint bounding} \rightarrow$$

$$\text{bbox}([\mathbf{p}_i] \cup [\mathbf{p}_{i+1}]) \oplus B_2(r) = \text{link swept-capsule bound}$$

that is the geometric core of RBF. The journal extension by Redon, Lin, Manocha, and Kim [9] applies the same framework to general robot arms and kinematic trees; that work has accumulated over 240 citations and is a standard CCD reference. Zhang *et al.* [10] replace interval arithmetic with Taylor models in the same pipeline to reduce wrapping errors, analogous to how RBF’s CritSample and analytical sources tighten the IFK bound. Schwarzer *et al.* [26] address a multi-query motivation similar to RBF’s—reusing collision evidence across planning calls—but at the path-segment level rather than as volumetric C -space records. Corrales *et al.* [27] apply sphere-swept line (SSL) bounding volumes dynamically for human-robot safety monitoring using the same structural observation that bounding a capsule’s motion reduces to bounding its two endpoints.

D. Novelty Boundary: Prior CCD vs. RBF

The two-step endpoint-interval-to-swept-capsule pipeline is therefore unambiguously present in the CCD literature for time-step motion. The differences that motivate the RBF contribution claim are: (a) the parameter domain is a *C-space joint-box interval* rather than a trajectory time step, so the goal is to classify whether *any* configuration in the box is collision-free, not whether a single given path collides; (b) the endpoint source is a switchable interface—IFK (conservative), CritSample (practical), and analytical extrema (tightest)—rather than a fixed algorithm, allowing the conservative vs. performance trade-off to be controlled without changing the link-representation or planner layers; (c) compact per-representation link records (AABB, KDOP26, SupportHull) are organised for staged filtering during box growth, a knob absent from CCD libraries; and (d) the endpoint and link envelope records are organised in LECT, a persistent *kinematic-keyed* cache that reuses evidence across multi-query builds in related scenes, a reuse level not addressed by CCD libraries or path-segment caches. No prior work has been found that applies the two-step endpoint bounding pipeline to *C-space* box growth or to multi-query volumetric planning reuse. RBF is complementary to the CCD stack: link envelopes reduce repeated geometric work on candidate boxes, while discrete scene checks remain the final authority for the reported paths.

E. Multi-Query Reuse and Adaptive Configuration-Space Decomposition

Multi-query planners typically amortize effort by reusing a graph structure such as a roadmap. PRM is the classical example: build a connectivity graph once and reuse it for many start-goal pairs [6], [28]. LazyPRM, lazy edge checking, and certificate-based planners postpone or cache edge validation so that online search does not repeatedly evaluate the same discrete local connections [29], [30]. Experience-based and trajectory-library systems reuse prior solutions or local motion snippets, often as initialization or bias for a new query rather than as a conservative decomposition of a region of configuration space [31].

RBF operates at a different reuse level. It grows and reuses volumetric boxes rather than only paths, binary collision outcomes, or zero-volume edges. PRM-like methods reuse connectivity and must revalidate affected edges after scene changes. Convex-region planners reuse higher-quality regions, but those regions are usually coupled to the obstacle layout in which they were constructed.

The method also inherits the central idea of adaptive cell decomposition and subdivision planning: represent free space by cells, split cells where the current evidence is insufficient, and search the adjacency graph induced by the accepted cells [32], [33]. The difference is the operating regime. Classical cell-decomposition examples are often low-dimensional workspace or explicit *C-space*

constructions, whereas RBF applies the same adaptive-cell idea to high-dimensional manipulator *C-space* through a link-envelope oracle. Scene-specific obstacle filtering is layered on top of reusable kinematic envelope evidence, with LECT available only as a cache for expensive records. The trade-off is explicit: RBF accepts axis-aligned boxes and over-approximation in exchange for simple validation, adjacency maintenance, and repeated-query throughput.

F. Sampling-Based Planning and Trajectory Optimization

Sampling-based planners remain standard baselines when the goal is to solve a single query quickly. RRT-Connect is particularly effective in manipulator planning because bidirectional growth often succeeds with very little global structure [34]. Asymptotically optimal and informed variants such as RRT*, Batch Informed Trees (BIT*), and Fast Marching Tree (FMT*) improve path quality and search efficiency when more time is available [7], [35], [36], [37]. The Open Motion Planning Library (OMPL) has made these methods easy to deploy and benchmark in a common software stack [38], [39]. However, standard RRT variants produce single-use zero-volume evidence and typically restart from scratch on the next query.

By contrast, RBF uses rapidly exploring, adaptive expansion to sample and materialize *C-space* boxes from the envelope oracle, producing a volumetric corridor structure that can be queried repeatedly within the same environment. Trajectory optimizers address a different stage of the pipeline. CHOMP, STOMP, TrajOpt, and Gaussian-process motion planners refine an initial path by local descent or sequential convex optimization [40], [41], [42], [43]. RBF uses these methods as complementary points of comparison: its aim is to provide a reusable volumetric backbone on which graph search and local post-processing can operate.

III. Link Interval Envelopes

This section develops the main geometric primitive used by RBF: a map from a manipulator joint box to compact link interval envelopes. The goal is to make box validation cheap enough for repeated-query planning. A strict interval source gives a conservative validation option, while tighter sampled or support-based variants are practical filters whose reported paths are discharged by the common final audit. Thus the strongest free-region statement in this section is conditional on the conservative source and on paths remaining inside the validated box union; the main planning experiments report final-audited performance separately. The construction first bounds endpoint trajectories and then applies a constant-radius lift for rounded link geometry.

Here $\mathcal{Q}$ denotes the joint-limit box explored by RBF, $I \subseteq \mathcal{Q}$ one joint interval, and $\mathcal{O} \subset \mathbb{R}^3$ the obstacle set. Endpoint envelope $E_k(I)$ outer-bounds endpoint k , and link envelope $B_\ell(I)$ outer-bounds the swept volume of link ℓ over the same interval. Together they define the

reusable planning primitive $I \mapsto \{E_k(I)\} \mapsto B_\ell(I)$. When the endpoint source is itself conservative, the resulting link envelope is conservative as well; when a tighter empirical source is used, the same representation serves as a fast broad- or mid-phase filter before final scene auditing rather than as a standalone certificate.

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ denote the rigid transform induced by a configuration $q \in I$. For a reference body $C_0 \subset \mathbb{R}^3$, its pose at q is $C(q) = T(q)C_0$. We write $S_I(C_0)$ for the exact swept workspace set over I and $C_I(C_0)$ for its convex envelope. RBF usually works with the convex envelope because the downstream representations are AABBs, hulls, or fixed-direction slab bounds. The following construction combines two classical computational-geometry facts—capsule Minkowski decomposition and convex-hull generator bounding—in a new planning context. Both facts are used individually in swept-sphere collision libraries and in CCD work for articulated bodies [8], [9]; the novelty here is applying their combination to *C-space box validation and multi-query reuse* rather than to single-query time-step CCD.

Minkowski sum separation (radius isolation). For a capsule $C = S \oplus B_2(r)$, rigid motion commutes with the fixed Euclidean ball, giving exact two-stage sweep bounds:

$$S_I(C) = S_I(S) \oplus B_2(r), \quad C_I(C) = C_I(S) \oplus B_2(r).$$

RBF therefore bounds the zero-radius seed first and applies the link radius at the end. This identity is exact for capsules; for sharp polytopes it requires the shape to be a rounded offset body.

Convex-hull generator bound. For a convex body $P = \text{conv}(v_1, \dots, v_m)$, bounding each vertex trajectory by an outer set $\hat{\Gamma}_i(I)$ suffices to bound the swept polytope:

$$S_I(P) \subseteq \text{conv}\left(\bigcup_i \hat{\Gamma}_i(I)\right).$$

For a capsule link the seed is the center segment $[a, b]$, so only two endpoint trajectories need to be bounded.

Combining these two facts yields the practical recipe. For a capsule link $[a, b] \oplus B_2(r)$, RBF bounds only the two endpoint trajectories, forms their convex hull, and applies the radius lift:

$$\hat{C}_I([a, b] \oplus B_2(r)) = \text{conv}(\hat{\Gamma}_a(I) \cup \hat{\Gamma}_b(I)) \oplus B_2(r).$$

The numerical task reduces to computing reusable envelopes for two endpoint trajectories per link, fully determined by endpoint-trajectory enclosures plus the chosen downstream representation.

Proposition 1 (Endpoint envelopes induce link interval envelopes). *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I . If endpoint envelopes $E_a(I)$ and $E_b(I)$ satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then*

$$S_I(L) \subseteq \text{conv}(E_a(I) \cup E_b(I)) \oplus B_2(r).$$

Consequently, any outer representation of the right-hand side is a valid link interval envelope for the whole joint box I .

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, so $T(q)[a, b] \subseteq \text{conv}(\Gamma_a(I) \cup \Gamma_b(I))$ over the entire interval. The endpoint inclusions place this set inside $\text{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. $\square$

B. Endpoint Interval Envelopes

Figure 1 illustrates the AABB extraction pipeline. Endpoint interval axis-aligned bounding boxes (iAABBs) are computed by interval forward kinematics (IFK) or tighter certified AA-based variants, combined into a conservative zero-radius centerline envelope, and padded by the link radius before collision querying. The remaining numerical task is to outer-bound each endpoint trajectory over I by a reusable set $E_k(I)$. Endpoint envelopes are the common interface between kinematic propagation and the downstream link-envelope representations.

For endpoint k , let $\Gamma_k(I)$ denote its exact workspace trajectory over interval I . Any conservative set that contains $\Gamma_k(I)$ is an endpoint interval envelope. In the baseline implementation this envelope is simply an axis-aligned bounding box, because boxes are cheap to store, easy to combine, and directly reusable by all downstream link representations.

a) *Interval forward kinematics (IFK).*: In the public API used in this paper, `EndpointSource.IFK` is the spelling for the IFK_AA-backed certified endpoint source. It propagates affine-arithmetic workspace bounds along the DH chain and remains the cheapest certified endpoint source in the active paper track, but it can loosen on wide boxes or long chains because one evaluation still accumulates residual over-approximation across the full joint box.

b) *Hierarchical IFK (HIFK).*: HIFK further reduces wrapping by recursively bisecting the joint box and evaluating AA-FK at the leaves before hulling the child endpoint boxes. This is not the same as the unsplit IFK baseline: a true depth-1 HIFK evaluation already splits once and hulls two leaves, whereas IFK performs one unsplit AA-FK pass on the original box. The mechanism study in section VI-C reports two representative split depths, HIFK_3 and HIFK_5, to quantify how additional hierarchical depth trades runtime for tighter certified endpoint boxes.

c) *Incremental FK state reuse.*: Tree descent also carries a transient FK state: interval-valued prefix transforms $[T^{0:0}], \dots, [T^{0:n}]$ and the per-joint DH interval matrices used to build them. The full prefix chain is computed once at the root or after a restart; when only I_k changes, prefixes before k remain valid and only the suffix is recomputed. This short-lived state is not the persistent cache itself; it only keeps endpoint materialization cheap before LECT decides whether to retain the evidence.

d) *Critical-sample envelopes.*: The second source estimates the endpoint box from carefully chosen boundary

and interior samples. In practice these samples include interval endpoints, turning-point candidates near multiples of $\pi/2$, and a few midpoint or near-boundary probes. This often captures extrema that IFK wraps away, at modest extra cost. The limitation is completeness: separable per-joint samples do not certify coupled extrema. CritSample is therefore a performance-oriented source whose reported planning paths are discharged by the final scene audit unless an external certificate is added.

e) Analytical extrema.: The third source exploits the trigonometric structure of the kinematic chain to tighten endpoint-coordinate bounds in selected low-dimensional cases. In the current implementation this source is used primarily as an offline profiling and certification aid rather than as the default online planner source. It is therefore reported as part of the endpoint-source design space, while the main planning rows rely on the sources and audit protocol specified in section VI.

Taken together, these sources define a clear spectrum: IFK is the cheapest certified AA baseline, HIFK_3 and HIFK_5 offer progressively tighter certified outer bounds at increasing cost, CritSample is the cheapest tighter advisory bound, and the analytical source is the tightest but most expensive. Endpoint envelopes are the right intermediate abstraction: they are the state variable of the joint-box-to-envelope map, so the downstream link-envelope layer stays fixed while the upstream source changes with the certification and runtime budget.

C. Link Interval Envelopes

Once endpoint envelopes are available, the remaining design question is how to represent the swept link volume compactly for box growth, collision filtering, and optional reuse. For a capsule link, RBF does not reason about the thick link directly. It first bounds the zero-radius center segment between the two endpoint envelopes and then pads that centerline envelope by the link radius. A conservative link interval envelope $B_\ell(I)$ is any set that contains the full swept capsule over interval I . In the default box-based implementation this is written as $B_\ell(I) = B_\ell^\circ(I) \oplus \mathcal{R}(r_\ell)$, where $B_\ell^\circ(I)$ bounds the centerline sweep and the padding operator $\mathcal{R}$ becomes B_∞ for ABB envelopes.

The following representations share the same endpoint evidence but preserve different amounts of downstream geometric detail.

a) ABB envelope.: ABB envelopes are the simplest option. The ordinary LinkIAABB construction takes the two endpoint envelopes, wraps them in one box, and pads that box by the link radius:

$$B_\ell(I) = \text{bbox}(E_{\ell-1}(I) \cup E_\ell(I)) \oplus B_\infty(r_\ell).$$

This representation can be tightened by subdividing the reference center segment into S pieces, bounding each piece separately, and retaining the union of the padded sub-envelopes. Larger S preserves more directional variation along the link while keeping reconstruction cheap once endpoint envelopes are available. The main caveat is that

the subdivision must remain explicit: if all sub-envelopes are collapsed back into one global ABB, much of the gain disappears.

b) KDOP slab envelope.: A k -discrete-oriented-polytope (k -DOP) envelope stores lower and upper support values along a fixed set of directions. Here the 26-plane variant (KDOP26) uses the three coordinate axes, six face-diagonal axes, and four body-diagonal axes, for a total of 13 unsigned axes and 26 supporting planes. Endpoint boxes are projected onto these axes, padded by the link radius, and merged over each subsegment. Collision checking is correspondingly cheap: after the ordinary link ABB test, the slab test evaluates the configured support directions and exits on the first separating interval.

c) SupportHull envelope.: When the endpoint envelopes are convex, their convex hull preserves directional structure that a single ABB discards. SupportHull is the compact runtime realization of that idea: instead of storing an explicit dense hull, it keeps the two endpoint ABBs and the link radius as a support-function record. For a query direction d , the support point is the better of the proximal and distal ABB support points, and collision refinement against an obstacle ABB uses a fixed-size Gilbert–Johnson–Keerthi (GJK) narrow phase with the obstacle inflated by the link radius and safety margin. This retains much of the convex-hull directional fidelity while keeping the default record compact; a hybrid variant may additionally retain KDOP26 intervals as a mid-phase before GJK, but that is treated as a separate runtime/storage point rather than as the default SupportHull envelope.

In this way, the link-envelope layer is separate from the upstream kinematics solver. Sections III-A to III-C define a modular endpoint-to-link envelope map from joint-space boxes to reusable workspace filters. Once endpoint envelopes are available, the remaining design choice is the representation and the corresponding planning-time cost/tightness trade-off.

IV. Lifelong Envelope Cache Tree

The planner can materialize the envelopes of section III online. LECT is a secondary optimization for workloads in which repeated builds revisit similar joint boxes or use envelope representations expensive enough to replay. It stores scene-independent endpoint/link-envelope evidence keyed by robot kinematics and joint intervals. Scene-specific obstacle tests, box labels, graph connectivity, and final-audit outcomes remain outside the cache. This section therefore records only the cache interface used by the planner; the scientific claim of the paper does not depend on LECT providing broad cross-scene transfer.

A. Cache Key and Split Policy

LECT is a dynamically grown binary KD tree over the joint-limit box $\mathcal{Q}$. Each node represents one joint interval and stores interval bounds, split metadata, cached

endpoint evidence from which link envelopes can be reconstructed, and residency state used by the implementation. The useful property is that the stored evidence is *kinematics-keyed* rather than *scene-keyed*: later scenes may replay a materialized envelope record, but they must rerun their own scene-local obstacle tests. The current storage format identifies nodes by 64-bit breadth-first heap ids (0 at the root and $2p + 1, 2p + 2$ for children). To keep this structural id scheme safe, the implementation enforces a configurable maximum tree depth $D_{\max} \leq 64$, with default $D_{\max} = 64$.

The split policy is envelope-aware rather than purely geometric. For a parent interval I , let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint, and let $\mathcal{V}(I) := \sum_{\ell} \text{vol}(B_{\ell}(I))$ be the cumulative downstream envelope volume. The default best-tighten score chooses the split with the smallest worst-child envelope burden,

$$d^* = \arg \min_d \max \left\{ \sum_{\ell} \text{vol}(B_{\ell}(I_d^-)), \sum_{\ell} \text{vol}(B_{\ell}(I_d^+)) \right\}.$$

The implementation uses this score in a depth-synchronous schedule so that all nodes at the same depth share one split decision. Candidate dimensions are also filtered by FK reachability: splitting a joint beyond the last active frame that contributes to any link endpoint cannot tighten the endpoint envelopes and is skipped.

B. Runtime Use and Validation Boundary

LECT is queried by FINDFREEBOX, not by the graph search itself. Given a joint interval, the cache either replays endpoint evidence or materializes it online, then derives the requested link representation. Directional records are used as refinement evidence rather than as standalone free-space certificates: the runtime first evaluates padded link AABBs, then uses KDOP26 slabs or SupportHull/GJK only for survivors of the AABB test. When a node is split, child envelopes may refine the parent witness, but any compact parent record remains representation-dependent and is rechecked against the current obstacle set before a box can enter the forest.

C. Storage and Warm-Build Semantics

Persistence is selective. Endpoint evidence is the common reusable record, but simple IFK+AABB evidence can be cheaper to recompute online than to reload. KDOP26 and SupportHull records are stored only when their tighter filters are expected to amortize storage and lookup cost. Implementation details such as node arrays, residency management, and canonical coordinate choices are engineering choices at the implementation level. The reported cache claim is limited to replay of compatible kinematic evidence and does not rely on any additional symmetry-transfer speedup.

The prewarm phase follows the same separation. It materializes FK/link-envelope evidence only on the target

leaf layer and reconstructs upper layers by child-hull propagation during checkpoint. This avoids charging direct FK at multiple depths for the same cache and keeps parent witnesses tied to the leaf records that generated them. Exact leaf evidence and propagated child-hull evidence are both treated as envelope records by the replay path; provenance does not create a separate reuse or validation mode.

In the warm-build path, LECT reuses only kinematic envelope evidence. On a cache hit, the current link-envelope representation is reconstructed from stored endpoint records and then revalidated against the current obstacle set. A warm build still regrows and revalidates scene-specific RBF boxes and never treats a persisted box label as a free-space certificate. In section V, LECT serves only as the materialization layer; obstacle checking, occupancy metadata, adjacency bookkeeping, and regrowth reactions to changing $\mathcal{O}$ remain inside the box-region forest routines. For the Shelf+IIWA protocol, the external d23 cache is treated as read-only during evaluation: warm shelf rows disable active-database backfill and therefore cannot silently accumulate new online envelope evidence while being measured. Other experiments state their own writeback policy when they measure cache growth rather than fixed held-out replay.

V. RapidBoxForest Construction

This section describes the planning layer of RBF. The algorithm grows a scene-specific forest of C -space boxes and exports the forest as a graph for repeated corridor queries. The growth loop is RRT-like, but the state added to the data structure is a free region rather than a point or edge: samples select frontiers, the link-envelope oracle materializes a usable box, and accepted boxes extend a connected region graph. In adaptive cell-decomposition language, the boxes are cells, FindFreeBox is the local refinement and classification primitive, and the forest graph is the adjacency structure used for query search. The link interval envelopes of section III and the optional LECT cache of section IV serve this planner as collision-filtering and materialization tools.

Throughout this section, a validated box means a box accepted by the selected scene-validation policy. In strict conservative mode, paths that remain inside the validated box union inherit the corridor statement below. In all reported planning experiments, paths that leave this union through repair, smoothing, or external composition are charged where used and passed through the same final scene audit.

Conceptually, the build process has four stages. First, a seed-guided local oracle adaptively refines and classifies a C -space cell around a selected configuration. Second, the leaf-sweep mode constructs a shallow, reusable partition of the current scene and identifies conservative refinement domains. Third, repeated frontier expansion or restricted refinement grows validated cells into a connected forest, giving the method its region-valued RRT/PRM character. Fourth, consolidation coarsens redundant boxes before the

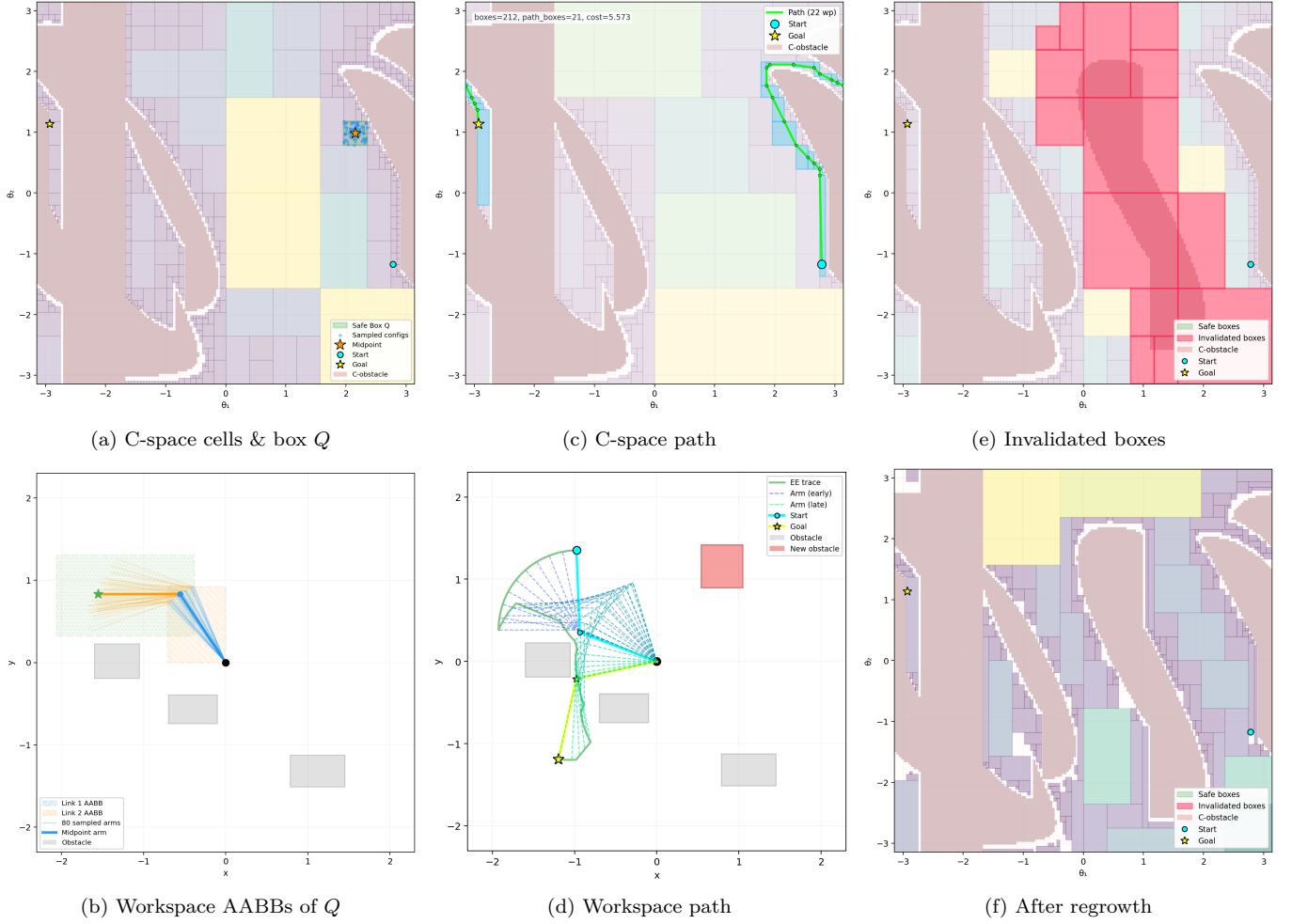


Fig. 2. Illustrative RBF build, query, and obstacle-update cycle on a planar 2-link manipulator. (a, b) Adaptive C -space cells and workspace AABB envelopes of the highlighted box Q . (c, d) Box-corridor path in C -space and its workspace trajectory. (e, f) After obstacle insertion, intersecting boxes are invalidated (red), and localized refill plus graph repair updates the explicit forest without requiring a full rebuild.

structure is exported as a query graph. The consolidation pass is part of the reusable build phase and its time is included in reported build measurements.

A. Construction Objective

Let $\mathcal{Q}$ denote the joint-limit box and let $\mathcal{O}$ denote the current obstacle set. The construction objective is to build a finite family

$$\mathcal{F} = \{B_1, \dots, B_N\}, \quad B_i \subseteq \mathcal{Q},$$

such that every B_i is a validated free-space box, the union $\bigcup_i B_i$ covers as much useful query-relevant free space as possible under the available box, time, and depth budgets, and the induced adjacency graph supports efficient downstream search. This object is scene-specific: changing $\mathcal{O}$ changes which intervals are accepted into the forest, even though the underlying envelope evidence in LECT remains reusable.

The central structural rule is that the forest is not merely a free-space cover, but a *corridor-compatible* free-space cover. Boxes are inserted under a connected-box constraint.

Viewed as adaptive cell decomposition, this objective is intentionally workload-biased rather than globally ex-

haustive. The planner does not try to classify every cell in $\mathcal{Q}$. It refines and accepts cells near useful frontiers, task seeds, and connector candidates, then keeps the resulting adjacency graph as the reusable planning object.

a) *Connected-box rule*.: For boxes

$$B_i = \prod_{d=1}^n [l_i^{(d)}, u_i^{(d)}], \quad B_j = \prod_{d=1}^n [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when, in every coordinate, the two intervals either overlap or are separated by at most a small tolerance ε_c :

$$\max\{l_i^{(d)}, l_j^{(d)}\} \leq \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \quad d = 1, \dots, n.$$

When $\varepsilon_c = 0$, this reduces to ordinary interval intersection or boundary touching in every dimension. A small positive tolerance absorbs the boundary offset used by grower seeds and floating-point roundoff.

This rule is deliberately weaker than requiring a full shared face. It treats overlap, touching, and tolerance-scale gaps uniformly as graph-connectivity evidence, while validity remains attached to the accepted box union itself. The output of the construction stage is a forest of validated boxes whose graph connectivity is aligned with the

Algorithm 1 FindFreeBox: local box materialization around a seed.

Input: Seed configuration q , LECT T , obstacle set $\mathcal{O}$, depth cap $D_{\max}$
Output: Validated interval I_v or failure

```

1:  $v \leftarrow \text{root}(T)$ ;  $d \leftarrow 0$ ; initialize current interval and FK state
2: while  $d \leq D_{\max}$  do
3:   Cache materialization
4:   materialize the endpoint and link envelopes of  $v$  if needed
5:   Scene validation
6:   if  $v$  is marked fully occupied then
7:     return failure
8:   end if
9:   if  $v$  is validated collision-free with respect to  $\mathcal{O}$  then
10:    return interval  $I_v$ 
11:   end if
12:   Refinement
13:   if  $d = D_{\max}$  then
14:     return failure
15:   end if
16:   if  $v$  has no children then
17:     split  $v$  along the depth-synchronous split dimension of depth  $d$ 
18:   end if
19:    $k \leftarrow$  split dimension of  $v$ 
20:    $v \leftarrow$  the child of  $v$  whose interval contains  $q$ ; update the current interval
21:   update the FK state from dimension  $k$  onward;  $d \leftarrow d + 1$ 
22: end while
23: return failure

```

corridor object queried later, without imposing a stricter face-sharing invariant than the grower actually maintains.

B. Find Free Box

FINDFREEBOX descends LECT from the root toward the leaf containing seed $q \in \mathcal{Q}$, materializing endpoint and link envelopes lazily along the path, skipping saturated subtrees via occupancy metadata, and returning the current interval as a validated box candidate as soon as the scene oracle accepts it. Tree topology and endpoint records are reused from the scene-independent cache, while scene-specific validation is deferred to each grower visit. Algorithm 1 gives the complete pseudocode.

The termination behavior is straightforward. Under depth cap $D_{\max}$, each loop iteration either returns immediately or descends one more level in the tree, so the procedure can make at most $D_{\max} + 1$ decisions. In strict conservative mode, any returned interval has passed envelope disjointness against $\mathcal{O}$; in performance-oriented modes, the returned interval is a validated planning region that remains subject to the common final audit used for reported paths.

C. Leaf Sweep and Restricted Refinement

The reported fast configuration uses FINDFREEBOX inside a two-stage leaf-sweep construction. Rather than asking the RRT-like grower to discover all useful coarse regions by random frontier proposals, RBF first enumerates a fixed-depth set of LECT leaves and classifies them with the current obstacle set. Let $\mathcal{L}_{d_0:d_1}$ denote the virtual leaf layer obtained by expanding the tree from start depth d_0 to maximum sweep depth d_1 . Each leaf $C \in \mathcal{L}_{d_0:d_1}$ is tested by the same link-envelope scene oracle used by

Algorithm 2 LeafSweepRefine: shallow coverage with restricted deep refinement.

Input: LECT T , obstacle set $\mathcal{O}$, sweep depths d_0, d_1 , refinement budget N_r
Output: Validated forest $\mathcal{F}$ and collision-domain cache $\mathcal{C}$

```

1:  $\mathcal{F} \leftarrow \emptyset$ ;  $\mathcal{C} \leftarrow \emptyset$ 
2: for all virtual leaves  $C \in \mathcal{L}_{d_0:d_1}$  do
3:   if  $C$  is certified free by the link-envelope scene oracle then
4:     insert  $C$  into  $\mathcal{F}$ 
5:   else
6:     record  $C$  and its blocker set  $\beta(C)$  in  $\mathcal{C}$ 
7:   end if
8: end for
9: build adjacency among boxes in  $\mathcal{F}$ 
10: for all collision domains  $C \in \mathcal{C}$  in priority order do
11:   generate interface, center, and boundary seeds inside  $C$ 
12:   for all seed  $q$  while budget remains do
13:      $B \leftarrow \text{FindFreeBoxInDomain}(q, T, \mathcal{O}, C)$ 
14:     if  $B \neq \emptyset$ ,  $B \subseteq C$ , and  $B$  is adjacent to  $\mathcal{F}$  or to a prior box from  $C$  then
15:       insert  $B$  into  $\mathcal{F}$  and update local adjacency
16:     end if
17:   end for
18: end for
19: return  $\mathcal{F}, \mathcal{C}$ 

```

FINDFREEBOX. A leaf that is certified free is inserted into the initial forest. A leaf that cannot be certified free is retained as a conservative *collision domain*. This domain is not an occupied proof; it is a bounded region in which the shallow test was inconclusive and where deeper refinement may still expose valid free boxes.

During this sweep, RBF also records a blocker set

$$\beta(C) \subseteq \{1, \dots, |\mathcal{O}|\}$$

for each non-free collision domain, containing the obstacles whose workspace AABs participate in the conservative overlap test. The blocker set is used only as update metadata. It is not used to certify free space, and it does not change the envelope validation rule for boxes that enter the forest.

After the shallow sweep, restricted refinement is performed only inside selected collision domains. Domains are ranked by their adjacency to already certified free boxes, volume, and proximity to query-priority configurations. Candidate seeds are placed at interfaces between a collision domain and neighboring free boxes, with additional center and boundary seeds to avoid depending on a single contact point. For a domain C , every accepted refined box B must satisfy

$$B \subseteq C$$

and must be adjacent either to an existing forest box or to a box already accepted from the same domain. Thus the refinement stage can fill narrow passages revealed inside a shallow collision domain without spending its budget on unrelated regions of $\mathcal{Q}$. The connector stage may subsequently add collision-checked segment witnesses between remaining islands, but these witnesses are recorded separately from box-overlap edges.

D. Forest Growth

Starting from an initial seed set, the forest is grown by repeatedly invoking the find-free-box procedure on

carefully chosen candidate configurations. Each seed first produces a root box through the find-free-box procedure, and these root boxes initialize one or more connected components of the forest. The main grower should be understood as an evidence-guided rapidly exploring process rather than as a generic random cover: every iteration chooses a frontier component, proposes a nearby interval worth testing, and accepts the result only when it extends the forest by a usable connected corridor segment.

Each iteration makes two explicit heuristic decisions: it samples a target configuration q_t , then chooses a boundary seed on an existing box face that is closest to that target. The reported shelf baseline uses a categorical allocation schedule rather than sequential Bernoulli gates. Its target masses are intertree-root $p_i = 0.25$, unexplored-volume $p_u = 0.20$, query-root $p_g = 0.15$, fixed anchor $p_a = 0.025$, and pure-uniform root sampling $p_{\text{uni}} = 0.375$; the remaining component-connector target probability is set to zero during the grow stage. This configuration is denoted `cat_i25_u20_r15_a025_localbest` in the experiment scripts. Build connectivity is then handled by the post-build connector, so segment bridges remain a fallback mechanism rather than the primary grower distribution. The same reported shelf configuration uses a d23 warm LECT cache, eight worker threads, a 40-level RBF evidence cap, a 15-level FFB start depth, a 28-level free-box cap for accepted boxes, 280 total boxes, 210 front-wave bootstrap boxes, and eight fixed IRIS/GCS anchor configurations. These numbers are planner settings rather than envelope-theory parameters, but they are part of the self-contained specification for the reported rows.

Given a target q_t and an existing box $B = \prod_j [\ell_j, u_j]$, the grower enumerates feasible faces $f = (B, d, s)$, where d is a joint dimension and $s \in \{-1, +1\}$ is the lower or upper side. Let τ be the adjacency tolerance and $\epsilon_b = \max(\epsilon_{\text{guard}}, \tau/4)$. A face is feasible only if stepping outside it by ϵ_b remains inside the global root interval. In the reported configurations $\epsilon_{\text{guard}} = 0$, so this formula uses the $\tau/4$ floor. The seed associated with the face is

$$q_f[j] = \begin{cases} u_d + \epsilon_b, & j = d, s = +1, \\ \ell_d - \epsilon_b, & j = d, s = -1, \\ \text{clip}(q_t[j], \ell_j + \epsilon_b, u_j - \epsilon_b), & j \neq d, \end{cases} \quad (1)$$

with clipping to $[\ell_j, u_j]$ when a box is thinner than $2\epsilon_b$. Faces are ranked by the squared target distance

$$\sigma(f; q_t) = \|q_t - q_f\|_2^2, \quad (2)$$

and the implementation retains the 128 lowest-score face candidates across the scanned frontier boxes. It then tries them in increasing score order, skipping a candidate if the corresponding LECT leaf is temporarily cooled or if a component-connection cache has already recorded the same boundary seed. The first remaining candidate becomes the seed for FINDFREEBOX.

The candidate box is accepted only if FINDFREEBOX validates it and the new box is connected to its parent under the same adjacency predicate used by the query

Algorithm 3 GrowForest: budgeted expansion of a connected box-region forest.

Input: LECT T , obstacle set $\mathcal{O}$, seed set $\mathcal{S}$, box budget N_{max} , time budget t_{max}
Output: Validated box-region forest $\mathcal{F}$

```

1:  $\mathcal{F} \leftarrow \emptyset$ ; initialize connected components from  $\mathcal{S}$ 
2: for all  $q \in \mathcal{S}$  do
3:    $B \leftarrow \text{FindFreeBox}(q, T, \mathcal{O})$ 
4:   if  $B \neq \emptyset$  then
5:     insert  $B$  into  $\mathcal{F}$  as a root box
6:   end if
7: end for
8: while  $|\mathcal{F}| < N_{\text{max}}$  and time budget not exhausted do
9:   draw target  $q_t$ : connector with prob.  $p_c$ , query/intertree root with prob.  $p_g$  or  $p_i$ , unexplored LECT leaf with prob.  $p_u$ , otherwise uniform
10:  enumerate feasible faces  $f = (B, d, s)$  of frontier boxes and compute  $q_f$  and  $\sigma(f; q_t)$ 
11:  choose the first of the 128 lowest-score face seeds that is not cooled and not in the frontier-seed cache
12:  set  $(q_{\text{seed}}, B_{\text{near}}) \leftarrow (q_f, B)$  for that face
13:   $B_{\text{new}} \leftarrow \text{FindFreeBox}(q_{\text{seed}}, T, \mathcal{O})$ 
14:  if  $B_{\text{new}} \neq \emptyset$  and  $\text{BoxesConnected}(B_{\text{new}}, B_{\text{near}}, \tau)$  then
15:    insert  $B_{\text{new}}$  into  $\mathcal{F}$ ;  $\text{UpdateComponents}(\mathcal{F})$ 
16:  else
17:    record the failed LECT leaf; cool it after one failed attempt for a 300-box horizon
18:  end if
19: end while
20: return  $\mathcal{F}$ 
```

graph: all dimensions overlap up to tolerance τ , and at least one dimension touches within tolerance or all dimensions overlap. Failures are also handled explicitly. When the LECT leaf containing a failed seed reaches the hard-frontier failure threshold of one failed attempt, the leaf is skipped for a horizon of 300 subsequently accepted boxes, unless a later depth stage raises the active search depth and enables a retry. Cooling and the frontier-seed cache do not alter the validation boundary; they only reduce repeated calls on already failed or already covered frontier regions. The resulting method is summarized in algorithm 3.

The main reported configuration uses coordinated multi-goal growth. Five seed configurations initialize the shelf roots, and the grower uses the fixed resource budget reported in section VI-A. This choice keeps the growth schedule simple enough to preserve one authoritative forest state while still exposing reusable LECT evidence across threads and cache-warm reruns. Staged wavefront growth and task-batched multi-threaded growth are treated as engineering variants, and the parallel-scaling experiment reports the present limit of the task-batched variant. The output of this section is the query-ready box-region object only.

E. Forest Consolidation

Raw growth tends to produce a forest that is finer than necessary. The purpose of consolidation is to replace locally valid boxes by fewer validated regions while preserving the graph structure used for corridor search. The implementation uses two conservative operations. First, exact connected pairs whose union is again a box may be merged after validation. Second, more general connected candidate pairs are ranked by a volume-expansion score

Algorithm 4 ConsolidateForest: graph-preserving merging and pruning.

Input: Validated forest $\mathcal{F}$, LECT T , obstacle set $\mathcal{O}$

Output: Consolidated validated forest $\mathcal{F}'$

```

1:  $\mathcal{F}' \leftarrow \mathcal{F}$ 
2: Stage 1: validated connected/contact merging
3: merge exact connected pairs whose union is itself a single box
4: for all connected candidate pairs  $(B_i, B_j)$  in  $\mathcal{F}'$  do
5:   build a candidate parent region  $H$  and score its volume expansion
6:   if the chosen envelope representation of  $H$  is collision-free with respect to  $\mathcal{O}$  then
7:     replace  $B_i, B_j$  by  $H$ 
8:   end if
9: end for
10: Stage 2: containment pruning
11: remove only contained boxes whose adjacency and segment witnesses transfer to selected covering regions
12: return  $\mathcal{F}'$ 

```

and accepted only if the candidate hull passes the same scene oracle. A final containment-pruning pass removes boxes that are fully covered by accepted validated regions when their graph incidence can be transferred. The consolidation logic is summarized in Algorithm 4.

Validity follows directly: every accepted merge has itself passed the scene oracle. Containment pruning removes only boxes fully covered by a selected region that inherits their adjacency witnesses, preserving the corridor connectivity used by query search.

F. Corridor Query

After growth and consolidation, the forest is exported as a graph $G = (V, E)$ with one vertex per validated box. Two vertices are connected by an edge when their boxes satisfy the connected-box rule above. Lightweight querying is possible because this connected-box structure is enforced during growth and consolidation rather than repaired after graph export.

If disconnected adjacency islands remain after growth, an optional connector stage attempts targeted bidirectional sampling bridges between nearest frontier boxes of different components. A successful bridge is stored as a segment edge rather than as a claim that the two boxes overlap. This distinction is important for both the RBF query layer and the GCS negative controls: a segment edge is a collision-checked path witness between two boxes, whereas a GCS edge over convex regions requires feasible overlap of the regions themselves.

This pair $(\mathcal{F}, G)$ is the final planning object produced by the build stage. Querying it is intentionally lightweight. Given start and goal configurations q_s, q_g , the planner associates them with start-side and goal-side boxes in $\mathcal{F}$ and then searches G for a connecting box sequence. The query problem is reduced to box-corridor retrieval inside an already constructed free-space representation.

A shortest path on G yields a box sequence whose union defines a corridor between the two query endpoints. A geometric path is then recovered as a waypoint chain inside that corridor. In the experiments, this candidate is then passed through the same OMPL-style simplification/local post-processing and fixed-step final audit used

by the comparison pipelines. That post-processing can move the final reported path outside the source box union. Consequently, the box-corridor statement below applies only to contained paths before or after post-processing, while the reported planning success in the main curves is the common fixed-resolution audit outcome.

Theorem 1 (Conditional validated box-corridor path). *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a continuous path $\gamma : [0, 1] \rightarrow \mathcal{Q}$ satisfies $\gamma([0, 1]) \subseteq \bigcup_{B \in \mathcal{F}} B$, then every configuration on γ is collision-free with respect to $\mathcal{O}$.*

Proof. For any $s \in [0, 1]$, the containment assumption gives a validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$. By construction, validation of B means every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (section III), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. $\square$

a) *Reporting protocol.* Theorem 1 is a conditional statement: it covers path portions contained in a conservatively validated box union. It is not the acceptance rule for the main experimental comparisons. When OMPL simplification, local repair, smoothing, or external composition leaves that union, the path is accepted only if it passes the same fixed-resolution final scene collision audit used for the baselines. The main performance curves should therefore be read as post-processed, fixed-audit planning results rather than as evidence that every reported path satisfies the box-corridor containment hypothesis.

G. Dynamic Obstacle Updates

The explicit forest representation also supports local graph maintenance under obstacle edits. The update procedure assumes fixed robot kinematics and a fixed LECT topology/evidence cache; only the scene obstacle set and the scene-dependent forest labels change. The key distinction is asymmetric: removing obstacles can only make previously blocked regions easier to certify, whereas inserting obstacles may invalidate boxes that were previously accepted.

For obstacle deletion, the blocker sets $\beta(C)$ recorded during the leaf-sweep stage provide a conservative promotion rule. When obstacle indices R are removed, each cached collision domain C updates its blocker set to $\beta(C) \setminus R$ with the remaining obstacle indices shifted accordingly. If this set becomes empty, C is promoted to the forest without a new envelope or collision-validation query. This is sound with respect to the information recorded by the shallow sweep: the only recorded reasons for rejecting C have been removed. The promoted box is still subject to the same graph-adjacency rule as any other forest box, and adjacency is updated incrementally rather than by rebuilding the full graph.

Obstacle insertion uses the dual operation. Each added obstacle is first tested against the current free boxes.

Algorithm 5 DynamicUpdate: obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, graph G , collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)
Output: Updated forest, graph, and collision-domain cache

```

1: if  $\Delta^- \neq \emptyset$  then
2:   for all  $C \in \mathcal{C}$  do
3:     remove deleted obstacle indices from  $\beta(C)$  and remap surviving indices
4:     if  $\beta(C) = \emptyset$  then
5:       promote  $C$  to  $\mathcal{F}$  if it is not contained in an existing box
6:     end if
7:   end for
8:   incrementally connect promoted boxes to overlapping or touching forest boxes
9: end if
10: if  $\Delta^+ \neq \emptyset$  then
11:    $\mathcal{I} \leftarrow \{B \in \mathcal{F} : B \text{ conflicts with some obstacle in } \Delta^+\}$ 
12:   remove  $\mathcal{I}$  and incident graph/segment edges from  $(\mathcal{F}, G)$ 
13:   for all  $B \in \mathcal{I}$  do
14:     sweep leaves inside  $B$  for at most  $k$  relative levels, capped by  $D_{\max}$ 
15:     insert free swept leaves that are adjacent to the surviving forest
16:     cache still-blocked leaves with blockers in  $\Delta^+$ 
17:   end for
18:   incrementally connect newly inserted boxes and re-audit surviving segment edges
19: end if
20: if the requested query is still disconnected then
21:   optionally run endpoint recovery and collision-checked segment fallback
22: end if
23: return updated  $(\mathcal{F}, G, \mathcal{C})$ 

```

Boxes whose link envelopes intersect the new obstacle are removed together with incident graph edges and segment witnesses that no longer survive an edited-scene audit. Each invalidated box then becomes a bounded refill domain. The reported update path performs a local leaf sweep inside this domain for at most k relative LECT levels, capped by the original global maximum depth. Free leaves found by this local sweep are reinserted into the forest; still-blocked leaves are cached with a blocker set containing the new obstacle index. The insertion update deliberately does not run a bounded RRT/FFB regrow stage, so the measured cost reflects invalidation, local leaf-sweep refill, and graph maintenance. If the edited graph still cannot answer the query, a final endpoint recovery and segment connector may add collision-checked witnesses; this fallback is reported explicitly in table VI.

If no contained conservative corridor is recovered, no box-corridor guarantee attaches to that query. The construction still exports a concrete box-region planning object, and section VI evaluates its build cost, query cost, path quality, and success rate after the same post-processing and fixed-resolution final audit used for the other planners.

VI. Experiments

This section evaluates RBF as a multi-query planning system under the current evaluation protocol. Returned paths are compared only after the implemented query pipeline has completed its own post-processing and strict final audit.

a) *Scope.*: The main evidence consists of the current Shelf+IIWA leaf-refine ablation, the link-envelope representation sweep, and the dynamic-update capability comparison. A reduced random-scene study across IIWA, UR5, and Panda and a compact Shelf+IIWA cross-algorithm sanity check are also reported for completeness.

b) *Reporting Protocol.*: Planner success in this paper means strict path-audit success under the current query pipeline. Cache reuse is allowed only at the envelope-materialization layer: target builds still regrow and revalidate scene-specific boxes, and warm rows are never credited for persisted free-space labels. For the Shelf+IIWA baseline warm rows, the external d23 cache is read-only during evaluation and active-database backfill is disabled.

c) *Fixed Configuration.*: The RBF planning rows use canonical IFK_AA endpoint evidence, the reported validation and final-audit policy, and the adopted link-envelope family {LinkIAABB, KDOP26, SupportHull}. The Shelf+IIWA ablation uses the current leaf-refine protocol: a virtual leaf sweep from depth 8 to 14 followed by restricted depth-28 refinement. Its baseline warm rows use the read-only d23 external cache, while non-baseline comparison rows are run without external cache replay or active endpoint-evidence caching unless explicitly stated.

d) *Platform and Scenes.*: Planner rows use the worker count recorded in each result artifact. The Shelf+IIWA leaf-refine baseline uses eight worker threads and reports an explicit single-thread ablation row; the standalone link-envelope representation microbenchmark uses the batch parallelism recorded in its result artifact. Shelf+IIWA uses the canonical Marcucci scene with five fixed query pairs. The random-scene benchmark uses balanced easy/medium/hard scenes for IIWA, UR5, and Panda under the staged random-scene anytime protocol.

A. Shelf+IIWA Efficiency and Path Quality

Figures 3 and 4 and table I summarize the Shelf+IIWA leaf-refine ablation. The d23 SupportHull baseline at the box200 point is the fastest audited row, while the box500/d24 point removes all segment-edge contribution and shortens the audited route at higher planning cost. The time decomposition shows that this trade-off is dominated by the restricted refinement budget after the leaf sweep; the connector is significant but no longer the primary source of path quality. The no-cache SupportHull row remains strict-audit successful but is slower and produces a longer route under the same box200 refinement budget. The no-cache AABB row is close in runtime and path length, but uses slightly more segment length; the defensible representation conclusion from this optimized shelf row is therefore not universal SupportHull dominance over AABB. The robust mechanism evidence is d23 evidence replay, anchor-guided restricted refinement, and parallel virtual leaf validation. Disabling priority-anchor roots fails the two shelf transitions that cross the narrowest region, identifying anchor-root refinement as a required mechanism rather than a cosmetic tuning parameter.

TABLE I

Shelf+IIWA leaf-refine ablation under the strict-audit protocol. The baseline rows use read-only d23 external evidence replay; all non-baseline rows disable external evidence and active endpoint evidence cache. Planning time excludes query audit time.

Row	Factor	Audit	Plan ms	Leaf ms	Refine ms	Conn. ms	Length	Segment frac.	Boxes
Baseline d23 SH box200	baseline	8/8	175.7	21.8	74.6	50.5	12.760	0.021	1764
Baseline d23 SH box400	trade-off	8/8	259.5	20.9	142.6	66.1	12.760	0.021	1964
Baseline d23 SH box500/d24	trade-off	8/8	380.7	21.8	270.3	58.1	12.158	0.000	2064
No-cache SH box200	cache	8/8	257.1	27.5	86.1	112.6	13.363	0.119	1809
No-cache AABB box200	envelope	8/8	258.0	22.2	97.0	108.5	13.140	0.121	1810
No-cache SH single-thread	threads	8/8	353.2	125.9	84.8	112.3	13.363	0.119	1809
No-cache SH no anchor roots	anchors	0/8	247.2	26.7	95.7	95.0	—	—	1809
No-cache SH serial leaf	leaf parallelism	8/8	357.6	129.2	85.7	113.1	13.363	0.119	1809

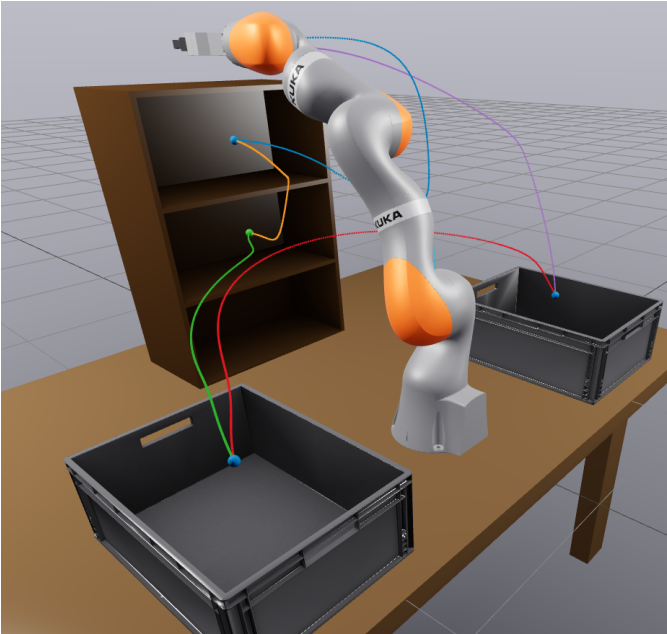


Fig. 3. Canonical Shelf+IIWA scene used in the experiments. The E3 main run evaluates five fixed query pairs on this scene under the strict audit protocol.

TABLE II

Shelf+IIWA cross-algorithm sanity check over three seeds and five fixed queries. Times follow each method’s artifact semantics: SBF reports median planning time excluding audit, PRM/BIT* report cumulative audited incumbent time, and RRTConnect reports query-only raw total time. Path length is now reported under a common success-only per-query mean: each seed first averages the final strict-audit-passing path lengths of its successful queries, then the table reports the median of those seed means.

Method	Audit	Time s	Mean Len.
SBF leaf-refine d23 box200	15/15	0.181	2.552
PRM shared roadmap, 5 s grid	13/15	16.236	3.556
BIT*, 5 s checkpoint	12/15	34.495	2.903
RRTConnect, 10 s timeout	15/15	0.169	4.298

Table II gives a compact cross-algorithm check on the same Shelf+IIWA task set. The updated SBF leaf-refine row succeeds on all 15 audited queries and is far faster than the PRM and BIT* incumbent traces under their cumulative accounting. RRTConnect also succeeds on all queries and is slightly faster as a one-shot query planner,

while the common per-query length column favors the SBF leaf-refine row on this shelf slice. This table is therefore not used to claim one-shot latency dominance over RRTConnect; its role is to separate the explicit audited box-region graph behavior from single-query sampling baselines. A reduced one-seed Shelf+IIWA IRIS-NP+GCS diagnostic with an eight-region prefix took 121.2 s and produced no audited solution on the five queries, so it is not treated as a competitive shelf row under the current configuration.

B. Balanced Random-Scene Pipeline Study

The multi-robot random-scene benchmark is a staged anytime generalization study, not the Shelf+IIWA leaf-refine protocol. It reports IIWA, UR5, and Panda across easy, medium, and hard settings using per-robot p18 warm caches and scene-specific staged SBF builds. This experiment is used to check that the audit and cache machinery run across robots and random scenes, while preserving the anytime structure of the older paper presentation. In fig. 5, SBF high-budget points are curve endpoints: they charge all preceding stages and mainly diagnose whether more box-forest budget buys path quality. They are not the fast random-scene baseline. Accordingly, table III reports the first strict-audit-successful SBF seed stage by difficulty; this removes the misleading impression that the 34 s high-stage endpoint is the baseline configuration. The reduced run passes strict audit across all nine robot–difficulty tasks for every reported row, but PRM, BIT*, and RRTConnect solve these one-shot random queries at much lower charged time than the staged SBF protocol, and the reduced IRIS-NP+GCS prefix is also faster than the SBF seed stage on this small random slice. The result should therefore not be read as a direct runtime extrapolation of the Shelf+IIWA leaf-refine baseline or as a claim that staged SBF dominates single-query sampling planners on random scenes.

C. Mechanism Studies: Link Envelopes

TABLE IV

Width-wise CritSample link-envelope comparison under the same $S = 4$ short-link split, including staged AABB/KDOP26/SupportHull cascades.

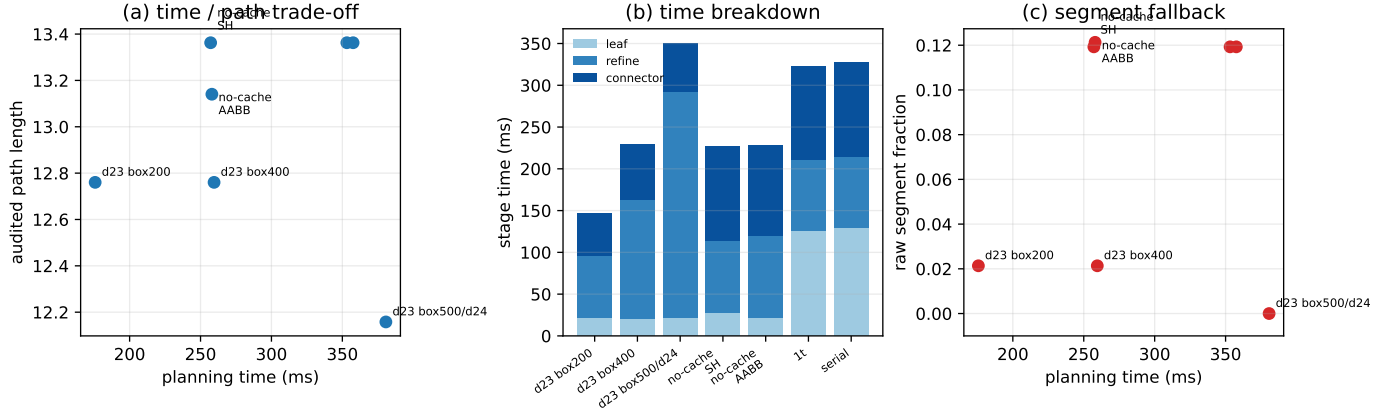


Fig. 4. Shelf+IIWA leaf-refine trade-off from the current Exp. 4 artifact. (a) Planning time versus final audited path length. (b) Stage-level planning time decomposition into virtual leaf sweep, restricted deep refinement, and connector. (c) Planning time versus raw segment-edge fraction before final OMPL simplification. The fast reported baseline is the d23 SupportHull box200/d28 point; the box500/d24 point is the zero-segment quality point.

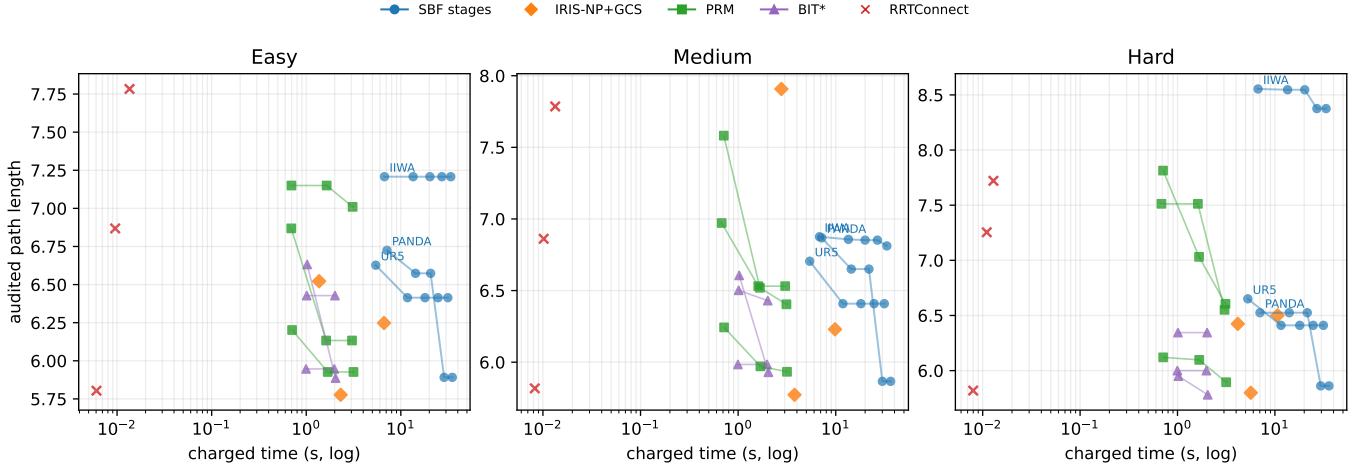


Fig. 5. Reduced balanced random-scene trade-off by difficulty. Each panel overlays IIWA, UR5, and Panda on one easy/medium/hard scene seed. SBF is shown as a cumulative staged curve from seed to high; PRM and BIT* are checkpoint curves; IRIS-NP+GCS and RRTConnect are single reported points in this reduced run. Points farther right on the SBF curve are additional quality-budget stages, not the selected fast baseline row in table III.

TABLE III

Reduced random-scene design-point summary by difficulty. Each difficulty aggregates one IIWA, one UR5, and one Panda task.

SBF uses the first strict-audit-successful seed stage; later SBF stages are shown in Fig. 5 as an anytime/quality curve rather than as the baseline row. Times follow each method’s artifact accounting. Len. is the strict-audit-passing incumbent mean path length under the same per-query convention for all methods.

Method	Stage	Audit	Median s	Easy		Medium		Hard	
				s	Len.	s	Len.	s	Len.
SBF seed stage	seed	9/9	6.818	6.817	6.725	6.928	6.868	6.818	6.651
IRIS-NP+GCS	r5	9/9	3.728	2.278	6.248	3.728	6.230	5.574	6.424
PRM	build1s	9/9	3.109	3.093	6.133	3.120	6.404	3.109	6.551
BIT*	t2s	9/9	2.011	2.011	5.947	2.010	5.984	2.013	6.000
RRTConnect	10s	9/9	0.0103	0.0097	6.868	0.0103	6.862	0.0112	7.254

Width	Envelope	V (m ³)	Payload	Eval (μs)
0.02	LinkIAABB	0.0017	384.0 B	0.5
0.02	KDOP26	0.0017	1.62 KB	1.4
0.02	SupportHull	0.0017	832.0 B	1.5
0.02	AABB->KDOP26	0.0017	646.1 B	0.7
0.02	AABB->SupportHull	0.0017	515.0 B	0.7
0.02	AABB->KDOP26->SupportHull	0.0017	777.1 B	0.9
0.05	LinkIAABB	0.0056	384.0 B	0.5
0.05	KDOP26	0.0056	1.62 KB	1.3
0.05	SupportHull	0.0056	832.0 B	1.1
0.05	AABB->KDOP26	0.0056	696.0 B	0.8
0.05	AABB->SupportHull	0.0056	540.0 B	0.7
0.05	AABB->KDOP26->SupportHull	0.0056	852.0 B	1.0
0.10	LinkIAABB	0.0229	384.0 B	0.7
0.10	KDOP26	0.0229	1.62 KB	1.6
0.10	SupportHull	0.0229	832.0 B	1.1
0.10	AABB->KDOP26	0.0229	758.4 B	1.0

TABLE V

Standalone link-envelope representation summary over the 2400-box fixed-width corpus. The table reports per-box staged materialization cost for LinkIAABB, KDOP26, SupportHull, and the AABB-first chain variants; the escalation column is the fraction of boxes that must run at least one post-AABB stage.

Rank	Representation	Payload	Build/box us
1	LinkIAABB	384.0 B	0.6809
2	KDOP26	1.62 KB	1.942
3	SupportHull	832.0 B	1.767
4	AABB->KDOP26	854.8 B	1.273
5	AABB->SupportHull	619.4 B	1.254
6	AABB->KDOP26->SupportHull	1.06 KB	1.847

a) *Link-envelope chain modes.*: Table IV extends the standalone link-envelope microbenchmark beyond independently built LinkIAABB, KDOP26, and SupportHull records. The staged chains run the cheap AABB filter first and invoke KDOP26 or SupportHull only for boxes that survive the earlier stage. Across the 2400-box fixed-width corpus, only 28.3% of boxes reach the tighter second stage, so the AABB-KDOP26 and AABB-SupportHull chains average 1.27 μ s and 1.25 μ s per box, compared with 1.94 μ s for standalone KDOP26 and 1.77 μ s for standalone SupportHull. The full AABB-KDOP26-SupportHull cascade averages 1.85 μ s per box. Table V compresses the same artifact into one aggregate summary view.

b) *Envelope sweep.*: Table V gives a compact cross-width summary of the representation study. LinkIAABB remains the cheapest standalone representation at 0.68 μ s per box and 384 B payload. Standalone KDOP26 and SupportHull increase cost to 1.94 μ s and 1.77 μ s, respectively. The AABB-first chains recover much of that overhead: AABB-KDOP26 averages 1.27 μ s per box at 854.8 B, AABB-SupportHull averages 1.25 μ s at 619.4 B, and all three chain rows escalate beyond the AABB stage on only 28.3% of the boxes in the fixed-width corpus.

D. Dynamic-Update Capability Study

Table VI reports the revised dynamic-update accounting for the leaf-sweep-RRT planner. The protocol uses saved constrained random AABB scenes sampled inside the Exp. 4 d23 canonical inverse-root domain, so the same scenes can be replayed for both incremental updates and warm target-scene rebuilds. Insertions scan the live free-box forest against the added obstacles and record invalidated boxes under those obstacles. Deletions query the shallow leaf-sweep collision-domain cache, promote domains whose blocker set disappears, and do not re-run envelope validation for those promoted domains. Insertions use a local relative-depth leaf-sweep refill inside invalidated boxes instead of a bounded RRT/FFB repair stage; Table VIII uses $k = 0$, i.e., it rechecks the invalidated leaf without descending deeper.

The result separates two qualitatively different regimes. For obstacle deletions, no accepted free box is invalidated; the blocker cache exposes about 7.1–8.1k candidate shallow domains per transition and the incremental update

is about 79–97 $\times$ faster than a fresh warm rebuild while preserving all eight queries. For obstacle insertions, the method correctly invalidates hundreds of free boxes and locally sweeps the invalidated domains. Four of eight insertion runs per transition require the final endpoint/segment fallback, but the total incremental update remains about 17–19 $\times$ faster than a warm rebuild and all audited queries succeed. This supports the intended use of segment connectivity as a last-resort recovery mechanism rather than the primary update operator.

VII. Conclusion

This paper presented RBF, a link-envelope-guided box-forest method for fast multi-query manipulator planning. Its geometric core maps endpoint interval envelopes to link interval envelopes via capsule Minkowski decomposition and convex-hull generator bounding; an RRT-inspired grower builds a scene-specific box forest into a corridor graph queried by graph search and local refinement; LECT optionally caches kinematic envelope records to reduce repeated materialization cost. The experiments show strict-audit Shelf+IIWA performance under the leaf-refine protocol, useful AABB-first link-envelope chain modes, and a cache-aware dynamic-update path with explicit invalidation and promotion accounting.

a) *Limitations.*: The box-corridor guarantee (theorem 1) is conditional on conservative IFK validation and path containment inside the validated box union; the reported rows do not claim that condition, and the dataset still does not store corridor-containment fields. The strict audit is an implementation-level acceptance rule rather than a continuous safety proof. The cache mechanism supports persistent evidence replay, but the reported results do not imply that replay produces wall-time speedup in every envelope setting.

b) *Scope and Future Work.*: The empirical scope covers Shelf+IIWA, link-envelope mechanisms, dynamic updates, and a reduced balanced random-scene pipeline check. Broader random-scene seed counts, stronger cache transfer, and dependency-complete IRIS/GCS comparisons remain natural next steps.

References

- [1] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.
- [2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, “Motion planning around obstacles with convex optimization,” *Sci. Robot.*, vol. 8, no. 84, 2023.
- [3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.
- [4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [5] —, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Tech. Rep. TR 98-11, 1998.
- [6] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.

TABLE VI

Leaf-sweep-RRT dynamic updates on constrained random AABB scenes sampled inside the Exp. 4 d23 canonical inverse-root domain. Update time excludes final query audit; warm is a fresh target-scene leaf-refine rebuild from the same read-only d23 evidence cache.

Transition	Edit	Update (ms)	Warm (ms)	Speedup	Invalid	Promoted	Cache cand.	Cache rej.	Repair boxes	Fallback	Islands	Seg.	Ok
easy->medium	insert	20.7	403.8	19.24	208	0	0	0	0	4/8	1	0.00	8/8
medium->hard	insert	20.8	432.0	17.25	313	0	0	0	0	4/8	1	0.00	8/8
hard->medium	delete	4.3	394.7	82.50	0	0	8054	0	0	0/8	1	0.00	8/8
medium->easy	delete	3.5	362.5	98.81	0	0	7108	0	0	0/8	1	0.00	8/8

- [7] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.
- [8] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, “Interactive and continuous collision detection for avatars in virtual environments,” in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.
- [9] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, “Fast continuous collision detection for articulated models,” *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.
- [10] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, “Continuous collision detection for articulated models using Taylor models and temporal culling,” *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.
- [11] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, “Finding and optimizing certified, collision-free regions in configuration space for robot manipulators,” in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.
- [12] M. Petersen and R. Tedrake, “Growing convex collision-free regions in configuration space using nonlinear programming,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.14737>
- [13] P. Werner, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10 359–10 365.
- [14] D. Blackmore and M. C. Leu, “Analysis of swept volume via Lie groups and differential equations,” *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.
- [15] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, “Swept volumes: foundation, perspectives, and applications,” *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.
- [16] S. Gottschalk, M. C. Lin, and D. Manocha, “OBBTree: A hierarchical structure for rapid interference detection,” in *ACM SIGGRAPH*, 1996.
- [17] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, “Fast proximity queries with swept sphere volumes,” University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.
- [18] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, “A fast procedure for computing the distance between complex objects in three-dimensional space,” *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.
- [19] G. v. d. Bergen, “A fast and robust GJK implementation for collision detection of convex objects,” *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.
- [20] J. Pan, S. Chitta, and D. Manocha, “FCL: A general purpose library for collision and proximity queries,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.
- [21] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.
- [22] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.
- [23] J. M. Snyder, “Interval analysis for computer graphics,” in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.
- [24] J.-P. Merlet, “Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis,” *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.
- [25] —, “Interval analysis for certified numerical solution of problems in robotics,” *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.
- [26] F. Schwarzer, M. Saha, and J.-C. Latombe, “Adaptive dynamic collision checking for single and multiple articulated robots in complex environments,” *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.
- [27] J. A. Corrales, F. A. Candelas, and F. Torres, “Safe human-robot interaction based on dynamic sphere-swept line bounding volumes,” *Robot. Comput.-Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.
- [28] R. Bohlin and L. E. Kavraki, “Path planning using lazy PRM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.
- [29] C. M. Dellin and S. S. Srinivasa, “A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors,” in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.
- [30] E. Schmerling, L. Janson, and M. Pavone, “Optimal sampling-based motion planning under differential constraints: The driftless case,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, “Reducing the barrier to entry of complex robotic software: A MoveIt! case study,” *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.
- [32] T. Lozano-Pérez, “Spatial planning: A configuration space approach,” *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.
- [33] R. A. Brooks and T. Lozano-Pérez, “A subdivision algorithm in configuration space for findpath with rotation,” *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.
- [34] J. J. Kuffner and S. M. LaValle, “RRT-connect: An efficient approach to single-query path planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
- [35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.
- [36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, “Informed sampling for asymptotically optimal path planning,” *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.
- [37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, “Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions,” *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.
- [38] I. A. Şucan, M. Moll, and L. E. Kavraki, “The open motion planning library,” *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.
- [39] M. Moll, I. A. Şucan, and L. E. Kavraki, “Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization,” *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.
- [40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.
- [41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, “STOMP: Stochastic trajectory optimization for motion planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.
- [42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, “Motion planning with sequential convex optimization and convex collision checking,” *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, “Continuous-time gaussian process motion planning via probabilistic inference,” *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.